Lab Report Submission

Course Code: CSE314

Course Name: Compiler Design Lab

Experiment No: 01

Experiment Name: Compiler Design Laboratory Experiments(Lab 01-04)

Submitted To

Name: Rabeya Khatun

Designation: Lecturer

Department: CSE

Daffodil International University

Submitted By

Name: MD. Emon Hossain

ID: 232-15-818

Section: 66_L2

Semester: Spring 2026

Department: CSE

Daffodil International University

Submission Date: 04/03/2026

www.diucoverpage.com

Index

Lab No.	Experiment Name	page
01	Basic String Operations in C	2-10
02	Lexical Analysis (Whitespace Identification, Removal & Tokenization)	11-18
03	Lexical Analysis (identifying tokens such as keywords, identifiers, operators, and constants.)	19-38
04	Regular Expressions	39-51

Experiment No. 01: Basic String Operations in C

1. Objective

The objectives of this experiment are:

- To understand the concept of strings in C programming
- To find the length of a string
- To copy one string into another string
- To concatenate two strings
- To compare two strings
- To reverse a given string

2. Pre-requisites

- Basic knowledge of C programming language
- Understanding of arrays and loops
- Familiarity with stdio.h and string.h libraries

3. Required Tools

- C Compiler (GCC / Dev-C++ / CodeBlocks / Turbo C)
- Any C IDE or text editor

4. Theory

In C programming, a **string** is a sequence of characters stored in a character array and terminated by a null character (\0).

The standard header file <string.h> provides several built-in functions for efficient string manipulation:

- strlen() – finds the length of a string
- strcpy() – copies one string to another
- strcat() – concatenates two strings
- strcmp() – compares two strings

String operations are widely used in system software, compilers, database systems, and general-purpose applications.

5. Programs

Note: Each operation is implemented using both **built-in functions** and **manual logic (without built-in functions)** to better understand internal string handling.

Program 1: Finding the Length of a String (Using strlen()) & without Built-in function.

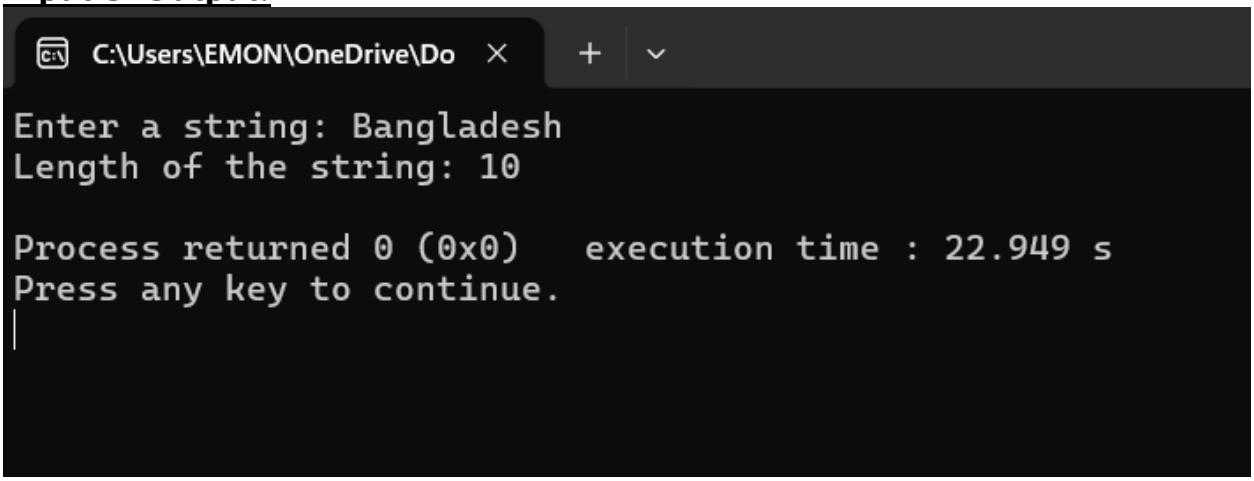
Code Without Built-in Function:

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[100];
    int length = 0;
    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);

    str[strcspn(str, "\n")] = '\0';

    for (int i = 0; str[i] != '\0'; i++) {
        length++;
    }
    printf("Length of the string: %d\n", length);
    return 0; }
```

Code (Using strlen()):

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[100];
    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);
    str[strcspn(str, "\n")] = '\0';

    printf("Length of the string: %lu\n",
strlen(str));
    return 0;
}
```

Input & Output:



Program 2: Copying One String to Another (Using strcpy()) & without Built-in function.

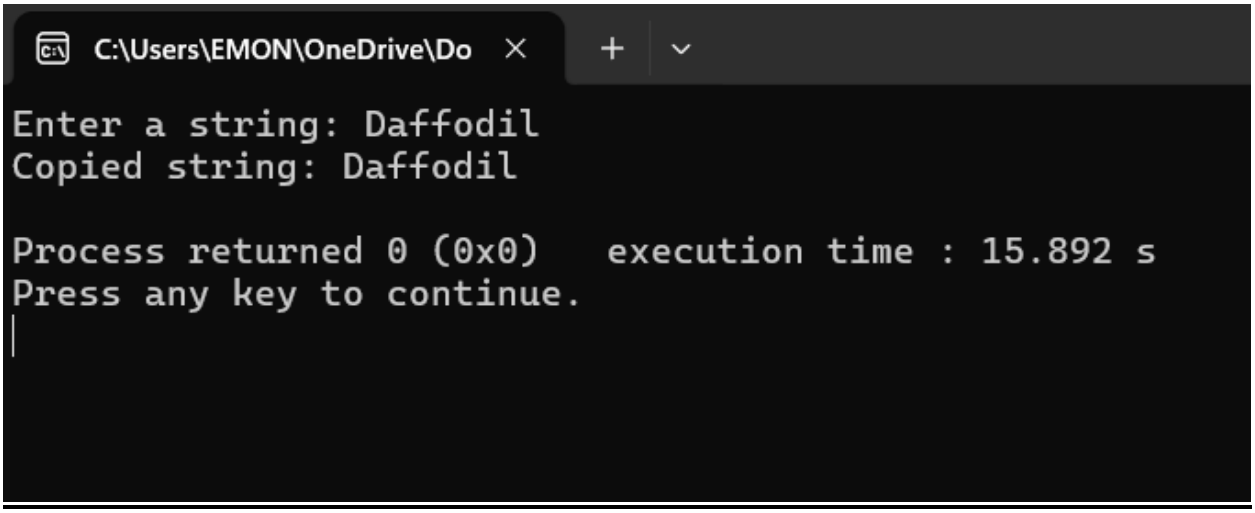
Without Built-in Function:

```
#include <stdio.h>
int main() {
    char str1[100], str2[100];

    printf("Enter a string: ");
    fgets(str1, sizeof(str1), stdin);

    for (int i = 0; str1[i] != '\0'; i++) {
        str2[i] = str1[i];
    }
    str2[i] = '\0'; // Null-terminate the copied
string

    printf("Copied string without using
built-in function: %s\n", str2);
    return 0;
}
```

Code(Using strcpy()):

```
#include <stdio.h>
#include <string.h>

int main() {
    char str1[100], str2[100];
    printf("Enter a string: ");
    fgets(str1, sizeof(str1), stdin);
    str1[strcspn(str1, "\n")] = '\0';

    strcpy(str2, str1);

    printf("Copied string: %s\n", str2);
    return 0;
}
```

Input & Output:



Program 3: Concatenating Two Strings (Using strcat()) & without Built-in function.

Without Built-in Function:

```
#include <stdio.h>
#include <string.h>

int main() {
    char str1[100], str2[100];
    int i, j;

    printf("Enter first string: ");
    fgets(str1, sizeof(str1), stdin);
    str1[strcspn(str1, "\n")] = '\0';

    printf("Enter second string: ");
    fgets(str2, sizeof(str2), stdin);
    str2[strcspn(str2, "\n")] = '\0';

    // Check buffer overflow
    if (strlen(str1) + strlen(str2) >=
sizeof(str1)) {
        printf("Error: Concatenation exceeds
buffer size.\n");
        return 1;
    }

    // Move i to the end of str1
    i = strlen(str1);

    // Append str2 to str1
    for (j = 0; str2[j] != '\0'; j++) {
        str1[i] = str2[j];
        i++;
    }

    // Null terminate the final string
    str1[i] = '\0';

    printf("Concatenated string: %s\n", str1);

    return 0;
}
```

Code(Using strcat()):

```
#include <stdio.h>
#include <string.h>

int main() {
    char str1[100], str2[100];
    printf("Enter first string: ");
    fgets(str1, sizeof(str1), stdin);
    str1[strcspn(str1, "\n")] = '\0';

    printf("Enter second string: ");
    fgets(str2, sizeof(str2), stdin);
    str2[strcspn(str2, "\n")] = '\0';

    strcat(str1, str2);
    printf("Concatenated string: %s\n", str1);
    return 0;
}
```

Input & Output:

```
C:\Users\EMON\OneDrive\Do

Enter first string: Bang
Enter second string: ladesh
Concatenated string: Bangladesh

Process returned 0 (0x0)   execution time : 37.780 s
Press any key to continue.
```

Program 4: Comparing Two Strings (Using strcmp()) & without Built-in function.

Without Built-in Function:

```
#include <stdio.h>
#include <string.h>

int main() {
    char str1[100], str2[100];
    int i = 0;
    printf("Enter first string: ");
    fgets(str1, sizeof(str1), stdin);
    printf("Enter second string: ");
    fgets(str2, sizeof(str2), stdin);

    while (str1[i] != '\0' && str2[i] != '\0') {
        if (str1[i] != str2[i]) break;
        i++;
    }

    if (str1[i] == str2[i])
        printf("Strings are equal.\n");
    else
        printf("Strings are not equal.\n");
    return 0;
}
```

Code(Using strcmp()):

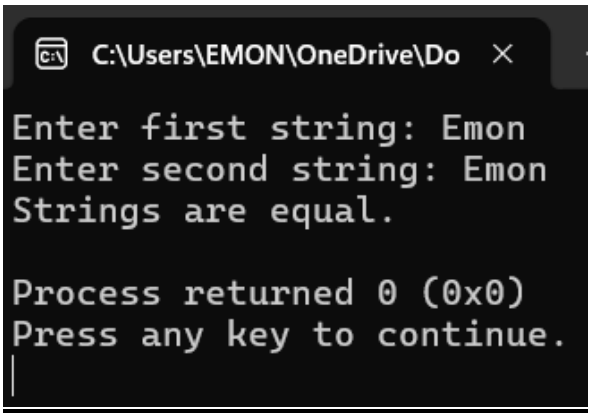
```
#include <stdio.h>
#include <string.h>

int main() {
    char str1[100], str2[100];
    printf("Enter first string: ");
    fgets(str1, sizeof(str1), stdin);
    str1[strcspn(str1, "\n")] = '\0';

    printf("Enter second string: ");
    fgets(str2, sizeof(str2), stdin);
    str2[strcspn(str2, "\n")] = '\0';

    if (strcmp(str1, str2) == 0)
        printf("Strings are equal.\n");
    else
        printf("Strings are not equal.\n");
    return 0;}
```

Alternative Solution:

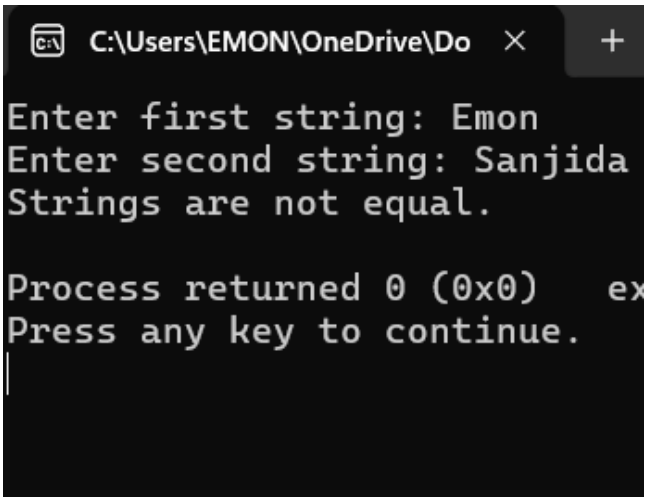
```
#include <stdio.h>
#include <string.h>
int main(){

char str1[100], str2[100];

printf("Enter first string: ");
fgets(str1, sizeof(str1), stdin);
str1[strcspn(str1, "\n")] = '\0';

printf("Enter second string: ");
fgets(str2, sizeof(str2), stdin);
str2[strcspn(str2, "\n")] = '\0';

int result = strcmp(str1, str2);

if (result == 0)
   printf("Strings are equal.\n");
else if (result < 0)
    printf("First string is less than second.\n");
else
    printf("First string is greater than
second.\n");

    return 0;
}
```

Input & Output:

Test Case-01:



Test Case-02:



Program 5: Reverse a string without using library functions & without using a second string.

Code Using Two strings and string length given by user:

```
#include <stdio.h>
#include <string.h>
int main() {

    int len;
    printf("Enter the max length of the string: ");
    scanf("%d", &len);
    getchar(); // clear newline

    char str1[len + 1];
    char str2[len + 1];

    printf("Enter a string: ");
    fgets(str1, sizeof(str1), stdin);
    str1[strcspn(str1, "\n")] = '\0';

    int actual_len = strlen(str1);

    int j = 0;
    for (int i = actual_len - 1; i >= 0; i--) {
        str2[j] = str1[i];
        j++;
    }

    str2[j] = '\0';

    printf("Reversed string: %s\n", str2);
    return 0;
}
```

Code Without using a second string:

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[100];
    int len, i;
    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);
    str[strcspn(str, "\n")] = '\0';

    len = strlen(str);
    for (i = 0; i < len / 2; i++) {
        char temp = str[i];
        str[i] = str[len - i - 1];
        str[len - i - 1] = temp;
    }
    printf("Reversed string: %s\n", str);
    return 0;
}
```

Input & Output:

```
C:\Users\EMON\OneDrive\Do

Enter a string: Bangladesh
Reversed string: hsedalgnaB

Process returned 0 (0x0)   execution time : 13.000 s
Press any key to continue.
```

6. Comparison: Built-in vs Without Built-in

Operation	Built-in Functions	Without Built-in
Length	Simple and fast	Logical and educational
Copy	Reliable and short	Manual memory handling
Concatenation	Clean syntax	Improves index control
Comparison	Accurate ASCII result	Easy exam logic
Reverse	Less code	Strengthens algorithm skills

7. Observations

- Built-in string functions reduce coding effort
- String comparison is case-sensitive
- Correct handling of the null character is essential

8. Result

All the objectives of this experiment were successfully achieved. The programs compiled and executed correctly, producing the expected outputs for each string operation such as length calculation, copying, concatenation, comparison, and reversal. Both built-in and manual implementations worked as intended, confirming a clear understanding of string manipulation concepts in C programming.

9. Conclusion

This experiment successfully demonstrated fundamental string operations in C programming. Understanding both built-in and manual implementations enhances problem-solving ability and prepares students for examinations and real-world programming tasks.

Experiment No. 02: Lexical Analysis (Whitespace Identification, Removal & Tokenization)

1. Objective

The objectives of this experiment are:

- To identify whitespace characters (space, tab, newline).
- To count whitespace characters in a given input.
- To remove whitespace from a string (using different methods).
- To tokenize input strings using the strtok() function.
- To understand the role of whitespace in lexical analysis.

2. Software Requirement

- C Compiler: MinGW
- Any C IDE: CodeBlocks
- Operating System: Windows

3. Theory

In compiler design, **lexical analysis** is the first phase of compilation. It scans the source code character by character and groups them into tokens.

Whitespace Characters

Whitespace characters include:

- Space ' '
- Tab '\t'
- Newline '\n'

These characters are used to separate tokens but usually have **no semantic meaning**, except inside string or character literals.

isspace() Function

The isspace() function (from <ctype.h>) checks whether a character is a whitespace character.

- Returns non-zero if true
- Returns 0 if false

strtok() Function

The strtok() function is used to split a string into tokens based on delimiters.

```
char *strtok(char *str, const char *delim);
```

It is commonly used in lexical analysis to separate identifiers, keywords, and symbols.

4. Algorithm

A. Whitespace Identification

1. Read a character.
2. Check if it is space, tab, or newline.
3. Display whether it is a whitespace character.

B. Whitespace Counting

1. Read a string.
2. Initialize a counter.
3. Traverse the string.
4. If isspace() is true, increment counter.
5. Print total count.

C. Whitespace Removal

1. Read input string.
2. Copy only non-whitespace characters to another string or same string.
3. Display the modified string.

D. Tokenization

1. Read input string.
2. Use strtok() with space as delimiter.
3. Print each token separately.

5. Program Implementation

Program 1: Identification of Whitespace (Without Library Function)

Code:

```
#include <stdio.h>
int main() {
    char a;
    printf("Enter a character: ");
    scanf("%c", &a);

    if (a == ' ' || a == '\t' || a == '\n') {
        printf("Whitespace character\n");
    }
    else {
        printf("Not a whitespace character\n");
    }
    return 0;
}
```

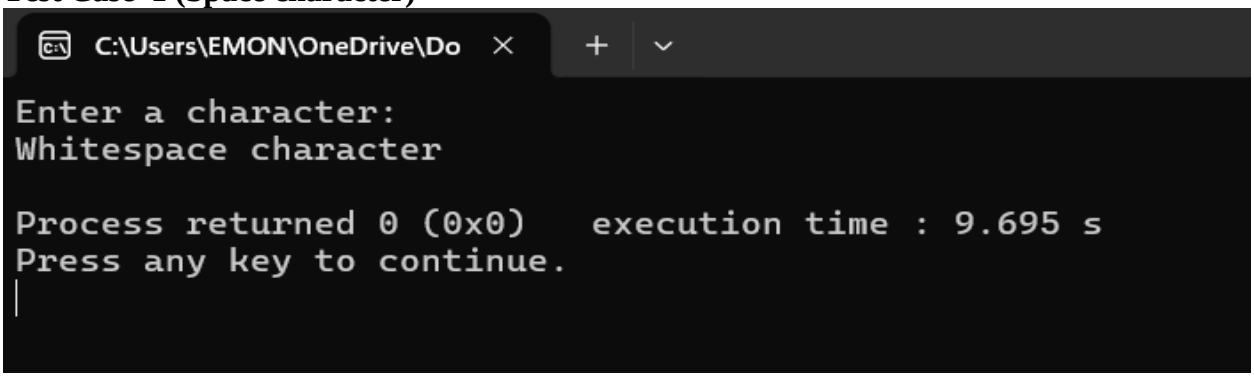
Program 2: Identification of Whitespace (Using isspace())

Code:

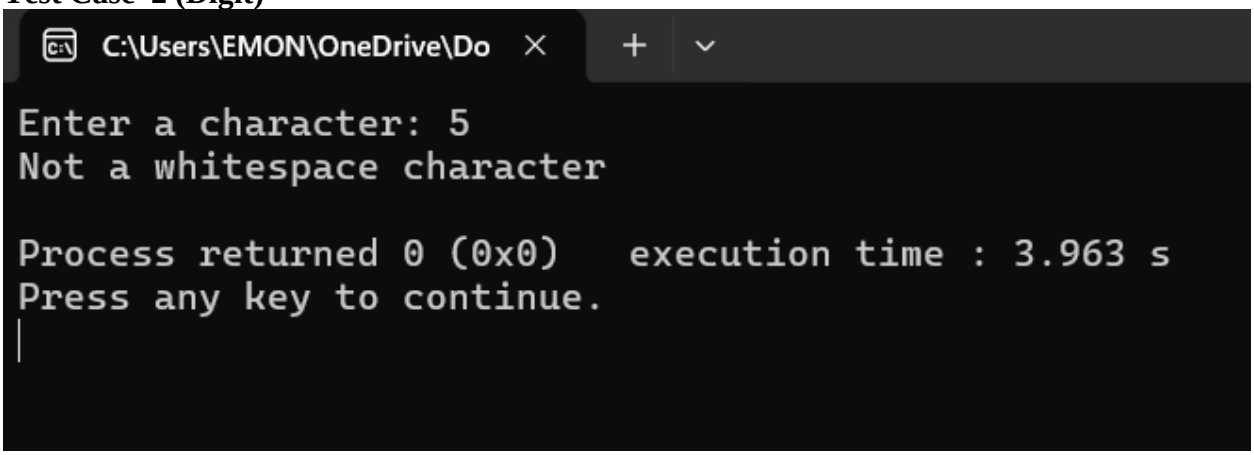
```
#include <stdio.h>
#include <ctype.h>
int main() {
    char a;
    scanf("%c", &a);

    if (isspace(a))
        printf("Whitespace character\n");
    else
        printf("Not a whitespace character\n");

    return 0;
}
```

Input & Output:

Test Case–1 (Space character)



Test Case–2 (Digit)



Comparison:

- Manual checking is basic and limited.
- **isspace()** is cleaner, readable, and handles all whitespace types.

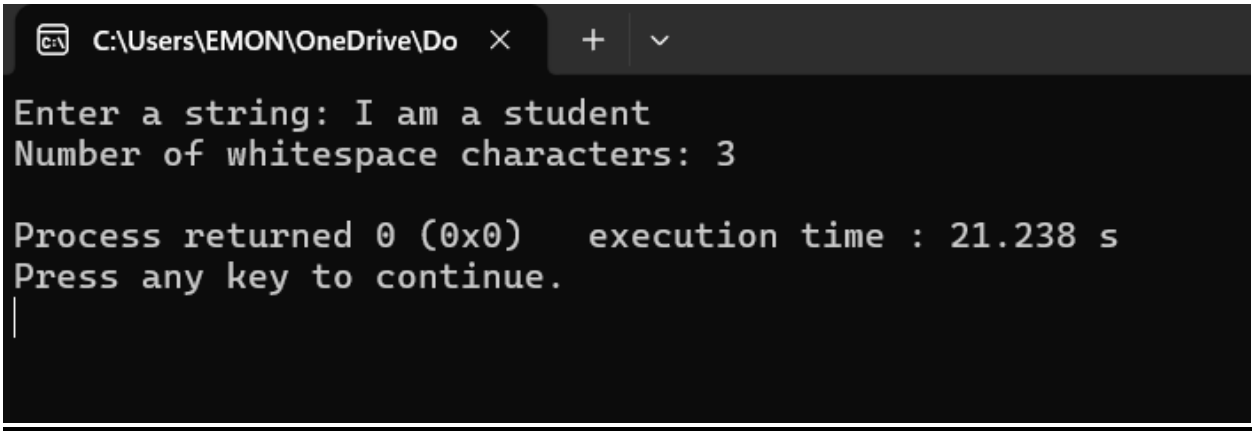
Program 3: Count Whitespace in a String

Code:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int main() {
   char str[100];
   int count = 0;

   printf("Enter a string: ");
   fgets(str, sizeof(str), stdin);
   str[strcspn(str, "\n")] = '\0';

   for (int i = 0; i < strlen (str); i++) {
      if (isspace(str[i]))
         count++;
   }
   printf ("Number of whitespace characters: %d\n", count);
   return 0;
}
```

Input & Output:

Test Case–1 (Sentence with spaces)



Program 4: Remove Whitespace Using Another String

Code:

```
 #include <stdio.h>
#include <string.h>
#include <ctype.h>

int main(){

    char str[100], result[100];

    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);
    str[strcspn(str, "\n")] = '\0';

    int j = 0;

   // for (int i = 0; i < strlen(str); i++)

   for (int i = 0; str[i] != '\0'; i++)
    {
        if (!isspace(str[i]))
        {
            result[j] = str[i];
            j++;
        }
    }

   result[j] = '\0';// Null-terminate the result string

    printf("String without whitespace: %s\n",
result);
    return 0;
}
```

Program 5: Remove Whitespace In-Place

Code:

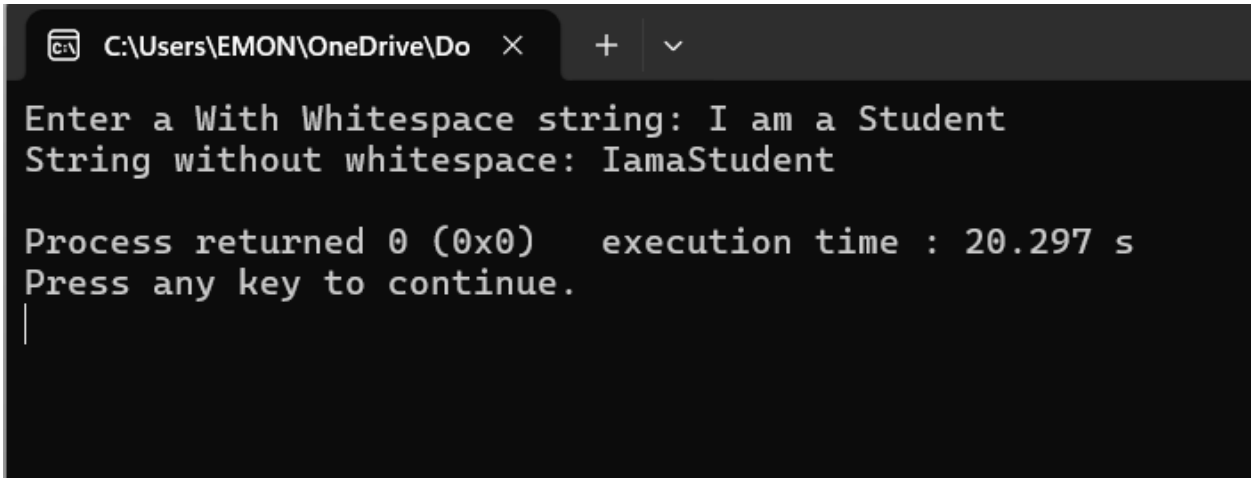
```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
int main() {

    int size;
    printf("Enter the size of string: ");
    scanf("%d", &size);
    getchar();

    char str[size];
    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);
    str[strcspn(str, "\n")] = '\0';
    int j = 0;

   /*
for (int i = 0; str[i] != '\0'; i++) {
        if (!isspace(str[i])) {
            str[j] = str[i];
            j++;
        }
    }
    */
    for (int i = 0; str[i] != '\0'; i++){
        if(isspace(str[i])){
            continue;
        }
        str[j++] = str[i];
    }
    str[j] = '\0';
    printf("String without whitespace: %s\n", str);
    return 0;
}
```

Comparison:

- Using a new string is safer and clearer.
- In-place modification is memory efficient.

Input & Output:

Test Case–1



Program 6: Tokenization Using strtok()

Code:

```
#include <stdio.h>
#include <string.h>
int main(){
   char str[100];


   printf("Enter a string: ");
   fgets(str, sizeof(str), stdin);
   str[strcspn(str, "\n")] = '\0';

   char *token;
   token = strtok(str, " ");

   //printf("first token: %s\n", token);//print first token

   while (token != NULL)
    {
      printf("%s\n", token);
      token = strtok(NULL, " ");
   }

   return 0;

}
```

Input & Output:

Test Case–1

```
C:\Users\EMON\OneDrive\Do

Enter a string: C programming is fun
C
programming
is
fun

Process returned 0 (0x0)   execution time : 5.679 s
Press any key to continue.
```

6. Comparison: Built-in Functions vs Without Built-in Functions

In lexical analysis, whitespace handling can be implemented either by **manual checking** or by using **built-in library functions**. Both approaches were explored in this experiment.

A. Whitespace Identification

Aspect	Without Built-in	Using Built-in (isspace())
Method	Manual comparison (' ', '\t', '\n')	Standard library function
Coverage	Limited to specified characters	Covers all standard whitespace
Readability	Less readable for large cases	More readable and clean
Reliability	Error-prone if cases are missed	More reliable

Using **isspace()** is preferable because it handles all whitespace characters and improves code clarity.

B. Whitespace Counting & Removal

Aspect	Without Built-in	Using Built-in
Logic Complexity	Higher	Lower
Code Length	Longer	Shorter
Maintenance	Difficult	Easy
Standard Compliance	Low	High

Built-in functions reduce complexity and follow standard C practices, making programs easier to debug and maintain.

C. Tokenization

Aspect	Manual Tokenization	Using strtok()
Implementation	Complex logic needed	Simple and efficient
Code Size	Large	Small
Error Possibility	High	Low
Performance	Moderate	Optimized

strtok() is highly efficient for lexical analysis and widely used in compiler design for token separation.

7. Result

All objectives of the experiment were successfully achieved. Different methods of identifying, counting, and removing whitespace were implemented and compared.

8. Conclusion

This experiment helped in understanding the practical implementation of lexical analysis concepts such as whitespace handling and tokenization, which are essential parts of compiler design.

Experiment No. 3: Lexical Analysis – Token & Number Identification

1. Objective

1. Identify keywords, identifiers, numbers, operators, and special symbols.
2. Verify the validity of identifiers.
3. Extract comments (single-line and multi-line) from source code.
4. Detect string literals in C source code.

2. Software Requirement

- C Compiler: MinGW
- Any C IDE: CodeBlocks
- Operating System: Windows

3. Theory

In compiler design, **lexical analysis** is the first phase of compilation. The **lexical analyzer (scanner)** reads the source code character by character and groups characters into **tokens**. Each token belongs to a token class, such as:

- **Keyword** – reserved word of the programming language **(e.g. int, float, if, else).**
- **Identifier** – user-defined name for variables, functions, arrays, etc. **(e.g. sum, count, var1).**
- **Number / Constant** – fixed numeric value **(e.g. 0, 123, -5).**
- **Operator** – symbol that performs an operation **(e.g. +, -, *, /, =, ==).**
- **Special symbol** – punctuation or structural characters **(e.g. ;, (), {}, [], ,).**

In this experiment, we build small C programs that perform specific lexical analysis tasks:

1. Check if a string consists only of digits → integer number.
2. Split a line of code into tokens and classify them using predefined lists.
3. Validate an identifier and a number using modular functions.

These programs demonstrate how simple conditions on characters (like isalpha(), isdigit(), etc.) can be combined to implement the token classification logic used inside a compiler.

4. Program Implementation

Program 1 – Identify Integer from String OR Identify Numbers

code without using library function

```
#include <stdio.h>
#include <string.h>
#include <stdbool.h>
int main() {
    char str[100];

    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);
    str[strcspn(str, "\n")] = '\0';

bool flag = 1;
    for (int i = 0; str[i] != '\0'; i++) {
        if (str[i] >= '0' && str[i] <= '9') {

          flag = 1;
        } else {
            flag = 0;
            break;
        }
    }
    if (flag) {
        printf("The string represents an
integer.\n");
    }
else {
        printf("The string does not represent an
integer.\n");
    }
Return 0;
}
```

code use isdigit() Library function

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
int main() {
    char str[100];


    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);
    str[strcspn(str, "\n")] = '\0';

    bool flag = 1;
    for (int i = 0; str[i] != '\0'; i++) {
        if (isdigit(str[i])) {
          flag = 1;
        } else {
            flag = 0;
            break;
        }
    }
    if (flag) {
        printf("The string represents an
integer.\n");
    }
 else {
        printf("The string does not represent an
integer.\n");
    }
  Return 0;
}
```

Input & Output:

Test Case–1

```
C:\Users\EMON\OneDrive\Do

Enter a string: 5
The string represents an integer.

Process returned 0 (0x0)   execution time : 10.329 s
Press any key to continue.
```

Test Case–2

```
C:\Users\EMON\OneDrive\Do

Enter a string: A
The string does not represent an integer.

Process returned 0 (0x0)   execution time : 4.204 s
Press any key to continue.
```

Program 2: Identify Keywords

Without function to check if the token is a keyword or not.

```
#include <stdio.h>
#include <string.h>
#include <stdbool.h>
int main(){
   char str[100];

   printf("Enter a string: ");
   fgets(str, sizeof(str), stdin);
   str[strcspn(str, "\n")] = '\0';

   char *token;
   token = strtok(str, " ");
   printf("first token: %s\n", token);

    char *keywords[] = {"if", "else", "while", "for", "return", "int", "float", "char", };


   while (token != NULL){
      int flag=0;
      for(int i=0;i<8;i++){
         if(strcmp(token,keywords[i])==0){
            flag=1;
            break;
         }
      }

      if(flag)
         printf("%s is a keyword\n",token);

      else
         printf("%s is unknown\n",token);


      token = strtok(NULL, " ");

   }
   return 0;
}
```

use function isKeyword() to check if the token is a keyword or not.

```
#include <stdio.h>
#include <string.h>
#include <stdbool.h>

const char *keywords[] = {"if", "else", "while", "for", "return", "int", "float", "char"};

int isKeyword(const char *token) {

   for (int i = 0; i < 8; i++) {
      if (strcmp(token, keywords[i]) == 0) {
         return 1;
      }
   }

   return 0;
}

int main() {
   char str[100];

   printf("Enter a string: ");
   fgets(str, sizeof(str), stdin);
   str[strcspn(str, "\n")] = '\0';

   char *token;
   token = strtok(str, " ");
   while (token != NULL) {
      if (isKeyword(token)==1) {
         printf("%s is a keyword\n", token);
      }
      else {
         printf("%s is unknown\n", token);
      }

      token = strtok(NULL, " ");
   }

   return 0;
}
```

Input & Output:

Test Case–1

```
C:\Users\EMON\OneDrive\Do

Enter a string: for
for is a keyword

Process returned 0 (0x0)   execution time : 4.201 s
Press any key to continue.
```

Test Case–2

```
C:\Users\EMON\OneDrive\Do

Enter a string: abbc
abbc is unknown

Process returned 0 (0x0)   execution time : 4.155 s
Press any key to continue.
```

Program 3 – Identify Identifier

Without function to check if the token is a Identifier or not.

```
#include <stdio.h>
#include <string.h>
#include <stdbool.h>
int main(){
    char str[100];
 printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);
    str[strcspn(str, "\n")] = '\0';

    char *identifiers[] = {"var1", "var2",
"count", "sum", "total"};
    char *token;
    token = strtok(str, " ");

     while (token != NULL){

        int flag_id=0;//false
        for(int l=0;l<5;l++){
            if(strcmp(token,identifiers[l])==0)//0-
means both are same
            {
                flag_id=1;
                break;
            }
        }
        if(flag_id){
            printf("The token '%s' is an
identifier.\n", token);
        }
        else {
            printf("The token '%s' is not an
identifier.\n", token);
        }

        token = strtok(NULL, " ");
        }
        return 0;
    }
```

use function isidentifier() to check if the token is a identifier or not.

```
#include <stdio.h>
#include <string.h>
#include <stdbool.h>
#include <ctype.h>

char *identifiers[] = {"var1", "var2", "count",
"sum", "total"};
int isidentifier(char *token) {
        int flag_id=0;//false
        for(int i=0;i<5;i++){
            if(strcmp(token,identifiers[i])==0)
                return 1;
        }
        return 0; // false
}
int main(){
    char str[100];
    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);
    str[strcspn(str, "\n")] = '\0';
    char *token;
    token = strtok(str, " ");
        while (token != NULL){
        if(isidentifier(token)){
            printf("The token '%s' is an
identifier.\n", token);
        }
        else {
            printf("The token '%s' is not an
identifier.\n", token);
        }

        token = strtok(NULL, " ");
    }
        return 0;

 }
```

Input & Output:

Test Case–1

```
C:\Users\EMON\OneDrive\Do

Enter a string: var1
The token 'var1' is an identifier.

Process returned 0 (0x0)   execution time : 6.509 s
Press any key to continue.
```

Test Case–2

```
C:\Users\EMON\OneDrive\Do

Enter a string: 1abc
The token '1abc' is not an identifier.

Process returned 0 (0x0)   execution time : 11.759 s
Press any key to continue.
```

Program 4 – Identify Valid Identifier OR Not

Code:

```
#include <stdio.h>
#include <ctype.h>
#include <stdbool.h>
#include <string.h>

int main() {
    char str[100];
    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);
    str[strcspn(str, "\n")] = '\0';  // remove newline

    bool isValid = false;

    // Empty string check - an empty string cannot be a valid identifier
    if (str[0] == '\0') {
        printf("The string is not a valid identifier.\n");
        return 0;
    }

    // 1. First character: must be letter or underscore
    if (isalpha(str[0]) || str[0] == '_') {
        isValid = true;
    } else {
        isValid = false;
    }
    if (isValid== true) {
        for (int i = 1; str[i] != '\0'; i++) {
            if (!(isalnum(str[i]) || str[i] == '_')) {
                isValid = false;
                break;
            }
        }
    }
    if (isValid) {
        printf("The string is a valid identifier.\n");
    } else {
        printf("The string is not a valid identifier.\n");
    }
    return 0;
}
```

Input & Output:

Test Case–1

```
C:\Users\EMON\OneDrive\Do

Enter a string: _ab9
The string is a valid identifier.

Process returned 0 (0x0)   execution time : 22.116 s
Press any key to continue.
```

Test Case–2

```
C:\Users\EMON\OneDrive\Do

Enter a string: 1a_
The string is not a valid identifier.

Process returned 0 (0x0)   execution time : 5.076 s
Press any key to continue.
```

Program 4 – Identify Operators & Special Symbol

For Operators

```
#include<stdio.h>
#include<string.h>
#include<stdbool.h>
#include<ctype.h>//ct
int main(){

    char str[100];
    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);
    str[strcspn(str, "\n")] = '\0';


    char *token;
    token = strtok(str, " ");

    char *operators[] = {"+", "-", "*", "/", "=",
"==", "!=","<", ">", "<=", ">="};

    while (token != NULL)
     {
        int flag_op=0;
       for(int j=0;j<9;j++){
          if(strcmp(token,operators[j])==0){
             flag_op=1;
             break;
          }
       }
       if(flag_op){
          printf("The token '%s' is an
operator.\n", token);
       }
       else {
          printf("The token '%s' is not an
operator.\n", token);
       }

       token = strtok(NULL, " ");
       }
       return 0;
    }
```

For Special Symbols

```
#include<stdio.h>
#include<string.h>
#include<stdbool.h>
int main(){{
    char str[100];
    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);
    str[strcspn(str, "\n")] = '\0';
    char *token;
    token = strtok(str, " ");
    char *special_symbols[] =
{";","()","{}",",","[]"};
    while (token != NULL)
     {
        int flag_sym=0;
       for(int m=0;m<5;m++){

if(strcmp(token,special_symbols[m])==0){
             flag_sym=1;
             break;
          }
       }

       if(flag_sym){
          printf("The token '%s' is a special
symbol.\n", token);
       }
       else {
          printf("The token '%s' is not a special
symbol.\n", token);
       }

       token = strtok(NULL, " ");

       }

       return 0;
}
}
```

Input & Output:

Test Case–1:For operator

```
C:\Users\EMON\OneDrive\Do

Enter a string: +
The token '+' is an operator.

Process returned 0 (0x0)   execution time : 3.860 s
Press any key to continue.
```

Test Case–2: For special Symbols

```
C:\Users\EMON\OneDrive\Do

Enter a string: ()
The token '()' is a special symbol.

Process returned 0 (0x0)   execution time : 3.772 s
Press any key to continue.
```

Program 5: Lexical Analyzer for Identifying Tokens in C Language

1.Design Lexical Analysis:

```
#include <stdio.h>
#include <stdbool.h>
#include <string.h>
#include <ctype.h>

const char *keywords[] = {
    "int", "char", "float", "while", "if", "for",
    "return", "double", "else", "do"
};

const char *operators[] = {
    "+", "-", "*", "/", "=", "<", ">", "<=", ">=", "++", "--"
};

const char *specialSymbols[] = {
    "{", "}", "#", "%", "&", "^", "(", ")", ",", "@"
};

bool isKeyword(char *str) {
    for (int i = 0; i < 10; i++) {
        if (strcmp(str, keywords[i]) == 0)
            return true;
    }
    return false;
}

bool isOperator(char *str) {
    for (int i = 0; i < 11; i++) {
        if (strcmp(str, operators[i]) == 0)
            return true;
    }
    return false;
}

bool isNumber(char *str) {
    for (int i = 0; str[i] != '\0'; i++) {
        if (!isdigit((unsigned char)str[i]))
            return false;
    }
    return (strlen(str) > 0);
}
```

```
bool isSymbol(char *str) {
    for (int i = 0; i < 10; i++) {
        if (strcmp(str, specialSymbols[i]) == 0)
            return true;
    }
    return false;
}

bool isIdentifier(char *str) {
    if (isalpha((unsigned char)str[0]) || str[0] == '_') {
        for (int i = 1; str[i] != '\0'; i++) {
            if (!isalnum((unsigned char)str[i]) && str[i] != '_')
                return false;
        }
        return true;
    }
    return false;
}

void removeSemicolon(char str[]) {
    int j = 0;
    for (int i = 0; str[i] != '\0'; i++) {
        if (str[i] != ';')
            str[j++] = str[i];
    }
    str[j] = '\0';
}

int main() {
    printf("========= Lexical Analyzer =========\n");
    printf("Type END to stop the program.\n\n");

    char str[100];

    while (1) {
        printf("Enter a valid statement:\n> ");

        if (fgets(str, sizeof(str), stdin) == NULL)
            break;

        str[strcspn(str, "\n")] = '\0';

        if (strcmp(str, "END") == 0) {
            printf("\nProgram Terminated.\n");
            break;
        }
```

```
        printf("\nOriginal String: %s\n", str);
        removeSemicolon(str);
        printf("After Removing Semicolons: %s\n", str);

        printf("\n------ Lexical Analysis Result ------\n");

        char *token = strtok(str, " ");

        while (token != NULL) {

            if (isKeyword(token))
                printf("Token: %s  -> Keyword\n", token);
            else if (isOperator(token))
                printf("Token: %s  -> Operator\n", token);
            else if (isNumber(token))
                printf("Token: %s  -> Number\n", token);
            else if (isSymbol(token))
                printf("Token: %s  -> Special Symbol\n", token);
            else if (isIdentifier(token))
                printf("Token: %s  -> Identifier\n", token);
            else
                printf("Token: %s  -> Unrecognized Token\n", token);

            token = strtok(NULL, " ");
        }

        printf("------------------------------------\n\n");
    }

    return 0;
}
```

Input & Output:

```
C:\Users\EMON\OneDrive\Do

========= Lexical Analyzer =========
Type END to stop the program.

Enter a valid statement:
> int a = 25 + b;

Original String: int a = 25 + b;
After Removing Semicolons: int a = 25 + b

------ Lexical Analysis Result ------
Token: int  -> Keyword
Token: a  -> Identifier
Token: =  -> Operator
Token: 25  -> Number
Token: +  -> Operator
Token: b  -> Identifier
-------------------------------------

Enter a valid statement:
>
```

2. Completion Task-week 03

```
// Simple Lexical Analyzer in C
// - Extracts comments
// - Extracts string literals
// - Classifies tokens: keyword, identifier, number, operator, special symbol

#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include <stdbool.h>

#define MAX 1000

const char *keywords[] = {
    "int","float","double","char","if","else","for","while","do","return",
    "void","switch","case","break","continue","default","long","short","unsigned",
    "signed","const","sizeof","struct","union","typedef","enum","static","extern",
    "register","volatile"
};

int num_keywords = 30;

bool isKeyword(char *word) {
    for (int i = 0; i < num_keywords; i++)
        if (strcmp(word, keywords[i]) == 0)
            return true;
    return false;
}

bool isValidIdentifier(char *str) {
    if (!isalpha(str[0]) && str[0] != '_')
        return false;

    for (int i = 1; str[i] != '\0'; i++)
        if (!isalnum(str[i]) && str[i] != '_')
            return false;

    return true;
}

bool isNumber(char *str) {
    int i = 0;

    if (str[0] == '+' || str[0] == '-')
        i = 1;
```

```
    for (; str[i] != '\0'; i++)
        if (!isdigit(str[i]))
            return false;

    return true;
}

bool isOperator(char ch) {
    char operators[] = "+-*/%=<>!&|^";
    for (int i = 0; operators[i] != '\0'; i++)
        if (ch == operators[i])
            return true;
    return false;
}

bool isSpecialSymbol(char ch) {
    char symbols[] = "();,{}[]#.";
    for (int i = 0; symbols[i] != '\0'; i++)
        if (ch == symbols[i])
            return true;
    return false;
}

void extractComments(char *str) {
    int i = 0;
    printf("\nComments found:\n");

    while (str[i] != '\0') {

        // Single-line comment //
        if (str[i] == '/' && str[i + 1] == '/') {
            i += 2;
            while (str[i] != '\0' && str[i] != '\n')
                printf("%c", str[i++]);
            printf("\n");
        }

        // Multi-line comment /* ... */
        else if (str[i] == '/' && str[i + 1] == '*') {
            i += 2;
            while (str[i] != '\0' && !(str[i] == '*' && str[i + 1] == '/'))
                printf("%c", str[i++]);
            if (str[i] != '\0')
                i += 2; // skip */
            printf("\n");
```

```
        }

        else {
            i++;
        }
    }
}

void extractStringLiterals(char *str) {
    int i = 0;
    printf("\nString Literals found:\n");

    while (str[i] != '\0') {
        if (str[i] == '"') {
            i++; // skip first "
            while (str[i] != '\0' && str[i] != '"')
                printf("%c", str[i++]);
            if (str[i] == '"')
                i++; // skip closing "
            printf("\n");
        } else {
            i++;
        }
    }
}

int main() {
    char code[MAX];

    printf("Enter C source code (end with EOF / Ctrl+D):\n");

    int c, idx = 0;
    while ((c = getchar()) != EOF && idx < MAX - 1)
        code[idx++] = (char)c;
    code[idx] = '\0';

    // First extract comments and string literals
    extractComments(code);
    extractStringLiterals(code);

    // Now tokenize and classify
    char token[MAX];
    int j = 0;

    printf("\nTokens and their types:\n");
```

```
    for (int i = 0; code[i] != '\0'; i++) {

        if (isalnum(code[i]) || code[i] == '_') {
            token[j++] = code[i];
        } else {
            // We hit a delimiter, so end of a word token
            if (j != 0) {
                token[j] = '\0';

                if (isKeyword(token))
                    printf("%s : Keyword\n", token);
                else if (isNumber(token))
                    printf("%s : Number\n", token);
                else if (isValidIdentifier(token))
                    printf("%s : Identifier\n", token);
                else
                    printf("%s : Invalid Identifier\n", token);

                j = 0;
            }

            // Check single character tokens
            if (isOperator(code[i]))
                printf("%c : Operator\n", code[i]);
            else if (isSpecialSymbol(code[i]))
                printf("%c : Special Symbol\n", code[i]);
        }
    }

    // Last token if any
    if (j != 0) {
        token[j] = '\0';
        if (isKeyword(token))
            printf("%s : Keyword\n", token);
        else if (isNumber(token))
            printf("%s : Number\n", token);
        else if (isValidIdentifier(token))
            printf("%s : Identifier\n", token);
        else
            printf("%s : Invalid Identifier\n", token);
    }

    return 0;
}
```

Input & Output:

```
C:\Users\EMON\OneDrive\Do

Enter C source code (end with EOF / Ctrl+D):
int main() {
  int a = 10;
  float b = 20.5;
  printf("Sum = %f", a+b);
}
^Z

Comments found:

String Literals found:
Sum = %f

Tokens and their types:
int : Keyword
main : Identifier
( : Special Symbol
) : Special Symbol
{ : Special Symbol
int : Keyword
a : Identifier
= : Operator
10 : Number
; : Special Symbol
float : Keyword
b : Identifier
= : Operator
20 : Number
. : Special Symbol
5 : Number
; : Special Symbol
printf : Identifier
( : Special Symbol
Sum : Identifier
= : Operator
% : Operator
f : Identifier
, : Special Symbol
a : Identifier
+ : Operator
b : Identifier
) : Special Symbol
; : Special Symbol
} : Special Symbol

Process returned 0 (0x0)   execution time : 8.367 s
Press any key to continue.
```

Experiment No. 04: Regular Expressions

1.Objective

The objectives of this experiment are:

1. To study the concept of Regular Expressions (RE).
2. To understand how RE is used in lexical analysis.
3. To implement C programs that recognize strings under given RE patterns.
4. To verify whether input strings match specific regular expressions.

2.Theory

3.1 Regular Expression (RE)

A Regular Expression (RE) is a sequence of characters that defines a search pattern.

It is widely used in:

- Lexical Analysis
- Pattern Matching
- Syntax Checking
- Token Identification

3.2 Basic Symbols of RE

Symbol	Meaning	Example	Matches
a	Character a	a	"a"
a*	Zero or more a	a*	"", a, aa
a+	One or more a	a+	a, aa
a	b	Either a or b	a
ab	Concatenation	ab	ab

4. Program Implementation

1.Program: Regular Expression (Lab Class)

Code:

```
 #include <stdio.h>
#include <string.h>

/* 1) a */
int is_a(const char *str) {
    return strcmp(str, "a") == 0;
}

/* 2) a|b */
int is_a_or_b(const char *str) {
    return (strcmp(str, "a") == 0) || (strcmp(str, "b") == 0);
}

/* 3) ab */
int is_ab(const char *str) {
    return strcmp(str, "ab") == 0;
}

/* 4) empty string (epsilon) */
int is_empty_string(const char *str) {
    return str[0] == '\0';
}

/* 5) a*  (includes empty string) */
int is_a_star(const char *str) {
    for (int i = 0; str[i] != '\0'; i++) {
        if (str[i] != 'a') return 0;
    }
    return 1;
}

/* 6) a+  (one or more a) */
int is_a_plus(const char *str) {
    if (str[0] == '\0') return 0;   // must have at least one char
    for (int i = 0; str[i] != '\0'; i++) {
        if (str[i] != 'a') return 0;
    }
    return 1;
}
```

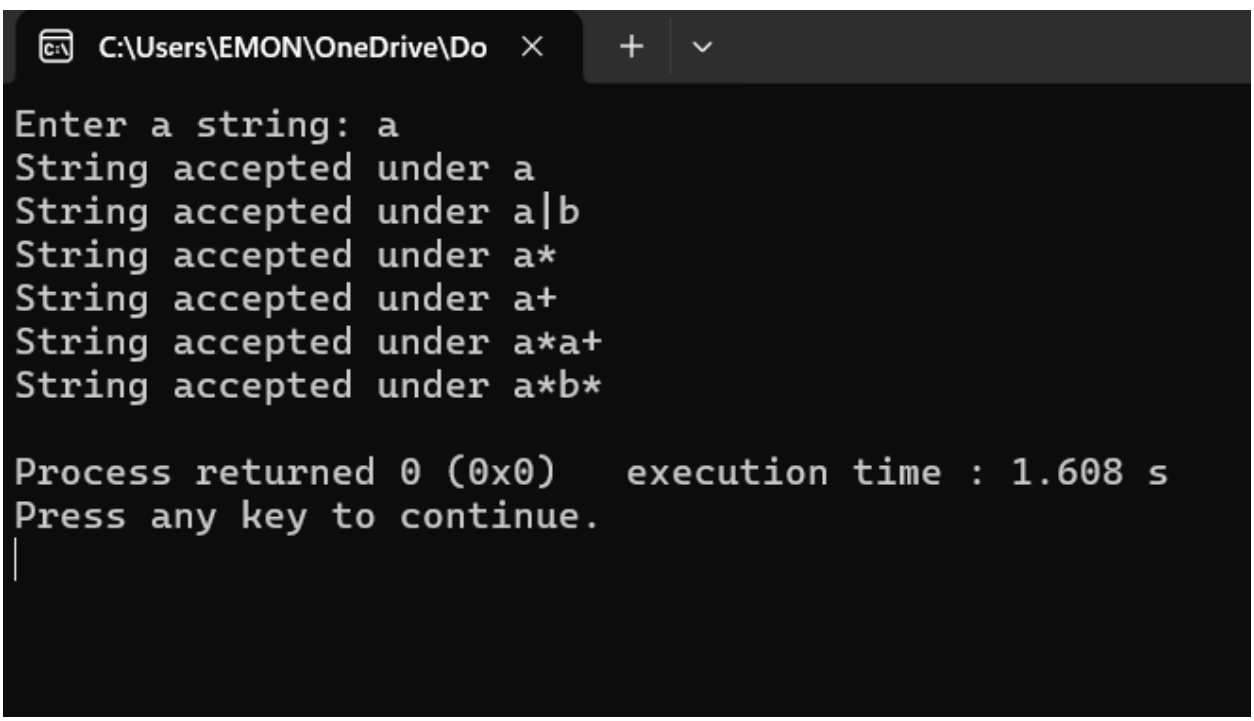
```
/* 7) a*a+  == a+ */
int is_a_star_a_plus(const char *str) {
    return is_a_plus(str);
}

/* 8) a*b+  (zero or more a, then one or more b) */
int is_a_b_plus(const char *str) {
    int i = 0;

    // consume a*
    while (str[i] == 'a') i++;

    // must have at least one b
    if (str[i] != 'b') return 0;

    // consume b+
    while (str[i] == 'b') i++;

    // must end here
    return str[i] == '\0';
}

/* 9) abb */
int is_abb(const char *str) {
    return strcmp(str, "abb") == 0;
}

/* 10) a*b*  (any number of a then any number of b, includes empty) */
int is_a_b_star(const char *str) {
    int i = 0;

    while (str[i] == 'a') i++;
    while (str[i] == 'b') i++;

    return str[i] == '\0';
}

/* 11) (a|b)+c  : one or more a/b then c at end */
int is_a_b_plus_c(const char *str) {
    int len = (int)strlen(str);
    if (len < 2) return 0;
    if (str[len - 1] != 'c') return 0;

    for (int i = 0; i < len - 1; i++) {
        if (str[i] != 'a' && str[i] != 'b') return 0;
    }
```

```
    return 1;
}

/* 12) (a|b)*c  : zero or more a/b then c at end */
int is_a_b_star_c(const char *str) {
    int len = (int)strlen(str);
    if (len < 1) return 0;
    if (str[len - 1] != 'c') return 0;

    for (int i = 0; i < len - 1; i++) {
        if (str[i] != 'a' && str[i] != 'b') return 0;
    }
    return 1;
}

/* 13) (ab)^2+  meaning: (ab)(ab)+  => "abab", "ababab", "abababab", ...  */
int is_ab_ab_plus(const char *str) {
    int len = (int)strlen(str);
    if (len < 4) return 0;
    if (strncmp(str, "abab", 4) != 0) return 0;

    // after "abab", the rest must be (ab)*
    for (int i = 4; i < len; i += 2) {
        if (i + 1 >= len) return 0;
        if (str[i] != 'a' || str[i + 1] != 'b') return 0;
    }
    return 1;
}

int main() {
    char str[100];
    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);

    // remove trailing newline
    str[strcspn(str, "\n")] = '\0';

    int matched = 0;

    if (is_a(str)) { printf("String accepted under a\n"); matched = 1; }
    if (is_a_or_b(str)) { printf("String accepted under a|b\n"); matched = 1; }
    if (is_ab(str)) { printf("String accepted under ab\n"); matched = 1; }
    if (is_empty_string(str)) { printf("String accepted under empty string\n"); matched = 1; }
    if (is_a_star(str)) { printf("String accepted under a*\n"); matched = 1; }
    if (is_a_plus(str)) { printf("String accepted under a+\n"); matched = 1; }
    if (is_a_star_a_plus(str)) { printf("String accepted under a*a+\n"); matched = 1; }
```

```
    if (is_a_b_star(str)) { printf("String accepted under a*b*\n"); matched = 1; }
    if (is_a_b_plus(str)) { printf("String accepted under a*b+\n"); matched = 1; }
    if (is_abb(str)) { printf("String accepted under abb\n"); matched = 1; }
    if (is_a_b_plus_c(str)) { printf("String accepted under (a|b)+c\n"); matched = 1; }
    if (is_a_b_star_c(str)) { printf("String accepted under (a|b)*c\n"); matched = 1; }
    if (is_ab_ab_plus(str)) { printf("String accepted under (ab)^2+\n"); matched = 1; }

    if (!matched) {
        printf("String not accepted\n");
    }

    return 0;
}
```

Input & Output:

Test Case–1:



Test Case–2:

```
C:\Users\EMON\OneDrive\Do

Enter a string: b
String accepted under a|b
String accepted under a*b*
String accepted under a*b+

Process returned 0 (0x0)   execution time : 2.071 s
Press any key to continue.
```

2.Lab Task-4: Experiment with Regular Expressions (completion Task)

```
#include<stdio.h>
#include<string.h>

//1. (a|b)*
int is_a_or_b_star( char *str) {
    for (int i = 0; i < strlen(str) ; i++) {
        if (str[i] != 'a' && str[i] != 'b') {
            return 0;
        }
    }
    return 1;
}

//2. ab*
int is_a_b_star( char *str) {
    if (strlen(str) == 0) {
        return 0;
    }
    if (str[0] != 'a') {
        return 0;
    }
    for (int i = 1; i < strlen(str); i++) {
        if (str[i] != 'b') {
            return 0;
        }
    }
    return 1;
}

//3. a+
int is_a_plus( char *str) {
    int count = 0;
    for (int i = 0; i<strlen(str) ; i++) {
        if (str[i] == 'a') {
            count++;
        }
        else {
            return 0;
        }
    }
    if (count > 0) {
        return 1;
```

```
    }


return 0;
}


//4. (ab|ba)
int is_ab_or_ba( char *str) {
    if (strlen(str) == 2 && str[0] == 'a' && str[1] == 'b') {
        return 1;
    }
    if (strlen(str) == 2 && str[0] == 'b' && str[1] == 'a') {
        return 1;
    }
    return 0;
}

//5. (ab)+
int is_ab_plus( char *str) {
    int len = strlen(str);
    if (len < 2 || len % 2 != 0) {
        return 0;
    }
    for(int i = 0; i < len; i += 2) {
        if(str[i] != 'a' || str[i + 1] != 'b') {
            return 0;
        }
    }

    return 1;
}


//6.a(bc)+
int is_a_bc_plus( char *str) {
    int len = strlen(str);
    if (len < 3 || str[0] != 'a' || len % 2 == 0) {
        return 0;
    }
    for (int i = 1; i < len; i += 2) {
        if (str[i] != 'b' || str[i + 1] != 'c') {
            return 0;
        }

    }
```

```
    return 1;
}


// 8. (a|b) c+ d
int is_a_or_b_c_plus_d( char *str) {

    int len = strlen(str);
    if (len < 3 || (str[0] != 'a' && str[0] != 'b') || str[len - 1] != 'd') {
        return 0;
    }
    for (int i = 1; i < len - 1; i++) {
        if (str[i] != 'c') {
            return 0;
        }
    }
    return 1;
}

// 9. (ab|ba)(ab)+
int is_ab_or_ba_ab_plus( char *str) {
    int len = strlen(str);
    if (len < 4 || ((strncmp(str, "ab", 2) != 0 && strncmp(str, "ba", 2) != 0)) || len % 2 != 0) {
        return 0;
    }
    for (int i = 2; i < len; i += 2) {
       if(str[i] != 'a' || str[i + 1] != 'b') {
            return 0;
        }
    }
    return 1;
}

//10. (a|b)*abb
int is_a_or_b_star_abb( char *str) {
    int len = strlen(str);
    if (len < 3 || str[len - 3] != 'a' || str[len - 2] != 'b' || str[len - 1] != 'b') {
        return 0;
    }
    for (int i = 0; i < len - 3; i++) {
        if (str[i] != 'a' && str[i] != 'b') {
            return 0;
        }
    }
    return 1;
}
```

```
//11. a(bc|cd)+
int is_a_bc_or_cd_plus( char *str) {
    int len = strlen(str);
    if (len < 3 || str[0] != 'a' || len % 2 == 0) {
        return 0;
    }
    for (int i = 1; i < len; i += 2) {
        if (!((str[i] == 'b' && str[i + 1] == 'c') || (str[i] == 'c' && str[i + 1] == 'd'))) {
            return 0;
        }
    }
    return 1;
}


//0(1|0)*
int is_0_1_star( char *str) {
    if (strlen(str) == 0 || str[0] != '0') {
        return 0;
    }
    for (int i = 1; i < strlen(str); i++) {
        if (str[i] != '0' && str[i] != '1') {
            return 0;
        }
    }
    return 1;
}

//12. (01|10)+

int is_01_or_10_plus( char *str) {
    int len = strlen(str);
    if (len < 2 || len % 2 != 0) {
        return 0;
    }
    for (int i = 0; i < len; i += 2) {
        if (!((str[i] == '0' && str[i + 1] == '1') || (str[i] == '1' && str[i + 1] == '0'))) {
            return 0;
        }
    }
    return 1;
}


int main() {
```

```
    char str[100];
    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);
    str[strcspn(str, "\n")] = 0;

        if (is_a_or_b_star(str)) {
            printf("The string matches the regular expression (a|b)*\n");
        } if (is_a_b_star(str)) {
            printf("The string matches the regular expression ab*\n");
        } if (is_a_plus(str)) {
            printf("The string matches the regular expression a+\n");
        } if (is_ab_or_ba(str)) {
            printf("The string matches the regular expression (ab|ba)\n");
        } if (is_ab_plus(str)) {
            printf("The string matches the regular expression (ab)+\n");
        } if (is_a_bc_plus(str)) {
            printf("The string matches the regular expression a(bc)+\n");
        } if (is_a_or_b_c_plus_d(str)) {
            printf("The string matches the regular expression (a|b) c+ d\n");
        } if (is_ab_or_ba_ab_plus(str)) {
            printf("The string matches the regular expression (ab|ba)(ab)+\n");
        } if (is_a_or_b_star_abb(str)) {
            printf("The string matches the regular expression (a|b)*abb\n");
        } if (is_a_bc_or_cd_plus(str)) {
            printf("The string matches the regular expression a(bc|cd)+\n");
        } if (is_0_1_star(str)) {
            printf("The string matches the regular expression 0(1|0)*\n");
        } if (is_01_or_10_plus(str)) {
            printf("The string matches the regular expression (01|10)+\n");
        } else {
            printf("The string does not match any of the specified regular expressions.\n");
        }

    return 0;
}
```

Input & Output:

Test Case–1:

```
C:\Users\EMON\OneDrive\Do

Enter a string: ab
The string matches the regular expression (a|b)*

Process returned 0 (0x0)   execution time : 11.029 s
Press any key to continue.
```

Test Case–2:

```
C:\Users\EMON\OneDrive\Do

Enter a string: a
The string matches the regular expression (a|b)*
The string matches the regular expression ab*
The string matches the regular expression a+
```

Test Case–3:

```
C:\Users\EMON\OneDrive\Do

Enter a string: abc
The string matches the regular expression a(bc)+
The string matches the regular expression a(bc|cd)+
```

Test Case–4:

```
C:\Users\EMON\OneDrive\Do

Enter a string: abcd
The string does not match any of the specified regular expressions.

Process returned 0 (0x0)   execution time : 8.727 s
Press any key to continue.
```

4.Result

The program was successfully implemented and tested.
The system correctly identifies whether input strings match the given regular expressions**.**

5.Conclusion

This experiment helped us understand:

- The concept of Regular Expressions.
- Pattern matching using C programming.
- Practical implementation of RE in lexical analysis.
- How compilers identify tokens using RE.

The experiment was successfully completed.

…….End…….